OPENING SESSION | EXTENDED TECHNICAL EDITION

is this IT?

From human computers to learning algorithms,
modern AI systems, and the engineers behind them.

MACHINE LEARNING CLUB • GRADES 6–12

TEACHING MAP

One deck. Three levels.

Use the first section for the opening club meeting; the remainder supports future technical workshops.

01 Discover

History • AI vs algorithms • features and labels • first model • class discussion

02 Solve

Regression • K-Means • trees • k-NN • perceptron • neural networks

03 Build

Evaluation • Python stack • scikit-learn • PyTorch • Transformers • agents

Suggested use First meeting: selected Discover slides + one exercise. Deep dives: one algorithm per week.

AUDIENCE QUIZ

Is this artificial intelligence?

Vote with your hands: AI, not AI, or depends. Be ready to justify your choice.

1. A calculator evaluates 27 × 14.2.
2. A spam filter learns from labeled emails.
3. A GPS app runs Dijkstra’s shortest-path algorithm.
4. A trained neural network recognizes a handwritten 9.

DEBRIEF

Automation ≠ learning

The essential distinction is whether a system learns from data.

Calculator: normally not ML

Executes known arithmetic rules.

Spam filter: ML

Fits a classifier on training examples.

Dijkstra: not ML by itself

A deterministic graph algorithm.

Digit recognizer: ML

Uses parameters learned from images.

Remember AI is broader than machine learning; not every automated tool is AI and not every AI system learns.

HISTORICAL FOUNDATION

The evolution arc

A narrative map, not a claim that every technology replaced what came before.

01

Human computation

People calculate

→

02

Programmable machines

Instructions + memory

→

03

Learning from data

Models are trained

→

04

Tool-using systems

Models can act

Key nuance Turing machines, the Bombe, CPUs, ML models, and agents solve different kinds of problems.

“Computer” was once a job title.

Scientific, engineering and military teams depended on human calculation.

Human computers

People produced numerical tables, verified calculations, and followed repetitive procedures.

Early machines

Mechanical and electromechanical equipment accelerated certain tasks but required human design and operation.

Ask students Which parts of this job are repetitive? Which require human judgment?

Two Turing stories—do not confuse them

1936 theoretical model versus 1940 wartime cryptanalysis.

Turing machine (1936)

An abstract computational model: read a symbol, change state, write, move. Used to study what algorithms can compute.

Turing–Welchman Bombe (WWII)

A physical electromechanical device that tested Enigma key hypotheses using logical implications from guessed plaintext “cribs”.

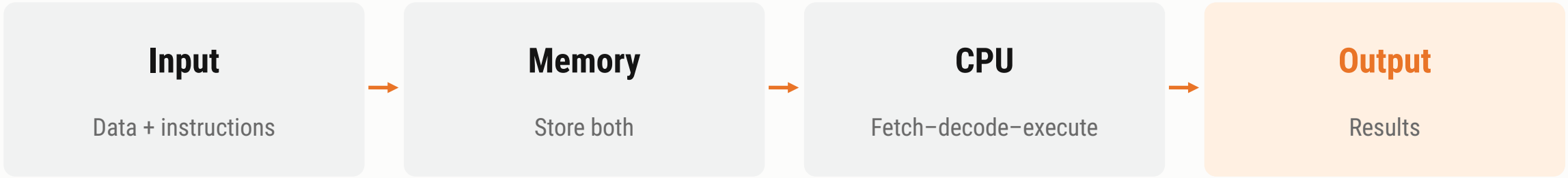
Not machine learning

The Bombe searched a constrained key space; it did not learn statistical parameters from labeled examples.

Source: The National Museum of Computing, “The Turing-Welchman Bombe”.

Why general-purpose computers changed everything

A program can be represented as data in memory: change the software, reuse the hardware.



Technical nuance The diagram is simplified: modern processors use caches, parallelism, interrupts, and multiple execution units.

A program can solve a problem without learning.

Start with a precise task, inputs, a finite sequence of steps, and an output.

Traditional algorithm

Input + rules chosen by a programmer → output

Example: binary search on a sorted list.

Machine learning

Training examples → learned parameters → predictions on unseen inputs.

Example: spam classification.

Question Can an algorithm be used inside a machine-learning system? Yes: training itself is algorithmic.

ALGORITHM CHALLENGE

Find 17 using binary search.

The list is sorted. Every comparison can eliminate about half the candidates.

PREDICT FIRST

[2, 5, 9, 12, 17, 20, 25]

Choose the middle element. After each guess, cross out an entire half.

PAIR WORK • 60 s

Binary search: worked trace

A sorted structure makes aggressive elimination possible.

Step	Low…high	Middle	Decision
1	[2, 5, 9, 12, 17, 20, 25]	12	17 > 12: go right
2	[17, 20, 25]	20	17 < 20: go left
3	[17]	17	Found

```
while low <= high:
    mid = (low + high) // 2
    if a[mid] == target: return mid
    if a[mid] < target: low = mid + 1
    else: high = mid - 1
```

Binary search: O(log n) comparisons, assuming random-access sorted input.

Can one program answer every question about programs?

No universal halting decider exists for arbitrary programs and inputs.

The Halting Problem (1936)

There is no algorithm that correctly determines whether every possible program will eventually halt on every supplied input.

UNSOLVABLE IN GENERAL

Caution This result does not mean that every individual program is impossible to analyze.

02 • LEARNING ERA

How machines learn

We replace some hand-written decision rules with parameters estimated from examples.

Three broad ways of learning

The label describes how training feedback is supplied, not a specific software library.

Supervised learning

Examples include inputs X and known targets y.

Spam labels, sale prices.

Unsupervised learning

No target label for each example. Seek structure in the available inputs.

K-Means grouping.

Reinforcement learning

An agent acts and receives rewards or penalties over time.

Game-playing policies.

Nuance The categories overlap with semi-supervised and self-supervised approaches; this is an introduction.

Datasets, examples, features, labels

We must decide what information the model can see and what outcome it should predict.

Song	Tempo (BPM)	Energy (0–10)	Mood label
A	65	3	Calm
B	72	2	Calm
C	142	9	Energetic
D	135	8	Energetic

Class exercise Features = tempo + energy. Label = mood. K-Means could instead learn groups without the mood labels.

ONE ESSENTIAL IDEA

Rules versus learned parameters

Compare prediction-time inputs to training-time information.

Traditional software

PROGRAMMER chooses explicit rules

Rules + new input → answer

Supervised machine learning

TRAINING ALGORITHM fits a model from examples

Trained model + new input → prediction

Technical nuance The algorithm is hand-designed; its fitted parameters are learned from training examples.

REAL ML PIPELINE

From question to useful prediction

The quality of the entire pipeline matters, not only the choice of algorithm.

01
Frame
Define target

→

02
Collect
Get examples

→

03
Prepare
Clean + split

→

04
Train
Fit model

→

05
Evaluate
Test on held-out data

→

06
Use
Monitor outputs

Iterative process After evaluation, return to the data, features, model, or objective as needed.

Data leakage: a model that “cheats”

Imagine predicting tomorrow’s rain using a column that records whether it rained tomorrow.

Leaked feature

“Actual rainfall tomorrow” is already part of the future target. It will make a model look excellent in training or evaluation.

Valid feature

Today’s pressure, temperature, or humidity is available before making tomorrow’s forecast.

Practical rule Never provide the model with information that would be unavailable at prediction time.

PEDAGOGICAL EVENT 01

Design a prediction problem

Pairs choose one target and only features measurable before the prediction.

PREDICT FIRST

Predict: will a student cycle to school tomorrow?

Suggest three potentially relevant features and one forbidden leaked feature.

THINK–PAIR–SHARE • 2 min

EXAMPLE SOLUTION

Which features are legitimate?

Prediction time determines what the algorithm is allowed to know.

Possible features

Weather forecast • planned timetable • distance to school

Forbidden feature

“How did the student actually travel tomorrow?” This is the target itself and is unknown at prediction time.

Important Even valid features can be intrusive. Collect the minimum data needed and respect privacy.

Predict a number with a line

Simple linear regression fits y-hat = w × x + b.

$$\hat{y} = w \cdot x + b$$

w = slope b = intercept

Ice-cream sales example

x: temperature (°C)
y: units sold

As temperature rises, sales often rise too. A line is a simple predictive hypothesis.

Fit and use a tiny linear model

Illustrative dataset follows y = x + 5 exactly (not realistic, but easy to check).

Temperature x	Observed sales y	Model $\hat{y} = x + 5$
10°	15	15
20°	25	25
30°	35	35
25° (new)	?	30

Solution The model predicts 30 sales at 25°C. Real examples will contain noise and prediction error.

How gradient descent changes a parameter

For one example x=2, y=6, use $\hat{y}=wx$ and squared loss $L=\frac{1}{2}(\hat{y}-y)^2$.

Start

w = 1

$\hat{y} = 2$ • $L = 8$

Slope

dL/dw = (wx−y)x

$(2-6)\times 2$ • −8

Update

w ← w − η·gradient

$\eta = 0.1$ • $w = 1.8$

Recheck

$\hat{y} = 1.8\times 2$

$\hat{y} = 3.6$ • $L = 2.88$

One update reduced loss from 8 to 2.88; more steps are often needed, and the learning rate matters.

Your turn: one gradient step

Use the same example, but a smaller learning rate.

PREDICT FIRST

$w = 1$, $x = 2$, $y = 6$, learning rate $\eta = 0.05$. What is the updated weight?

The gradient is -8. Apply $w_{new} = w - \eta \times \text{gradient}$.

PAIR WORK • 90 s

Gradient descent update

Both reasonable step sizes improve this specific single-example loss.

$$w_{new} = 1 - 0.05 \times (-8) = 1.4$$

New prediction: 2.8 New loss: 5.12

Reflection

A step can be too small or too large. Learning rate is a hyperparameter, not a magic number.

Learn a sequence of questions

A decision tree splits data on features to create smaller, more uniform groups.

Is it raining?

Feature question

→

YES → Umbrella

First branch

→

NO → Maybe

Another question

How it learns

Training chooses splits that reduce node impurity (or another fitting criterion).

How does a tree choose a split?

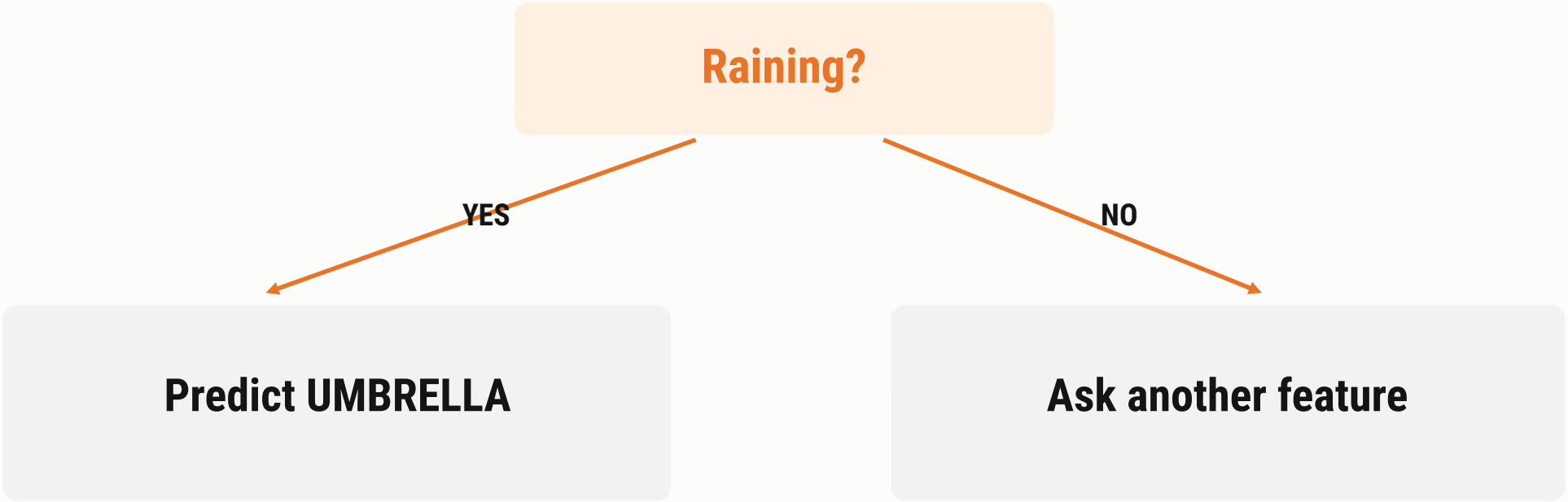
For a binary classification node, Gini = 1 − p(yes)² − p(no)².

Node	YES	NO	Gini impurity
Before split	4	2	$1-(4/6)^2-(2/6)^2 = 0.444$
Rain = yes	3	0	0
Rain = no	1	2	$1-(1/3)^2-(2/3)^2 = 0.444$

Weighted split $(3/6)\times0 + (3/6)\times0.444 \approx 0.222$. Impurity reduction ≈ 0.222.

Follow the learned tree to a prediction

A classification tree asks one question at each internal node.



Limits Deep trees may memorize training examples; limit depth, prune, or validate hyperparameters.

Learn by similarity, not by explicit rules

Store examples; predict from labels of the k closest points under a chosen distance.

Known point	Position x	Label
A	2	Low
B	3	Low
C	8	High
New point	6	?

Vote changes with k

k = 1 → closest is 8 → HIGH

k = 3 → neighbors 2, 3, 8 → LOW

Scaling and distance choice matter.

Clustering without target labels

Choose K centroids, assign each point to its closest centroid, then recompute centroids.

Initialize
Pick K centers

→

Assign
Nearest centroid

→

Update
Mean of each group

→

Repeat
Until stable

Unlike classification

Cluster IDs do not automatically correspond to human meanings such as “calm” or “energetic”.

Four points, two clusters

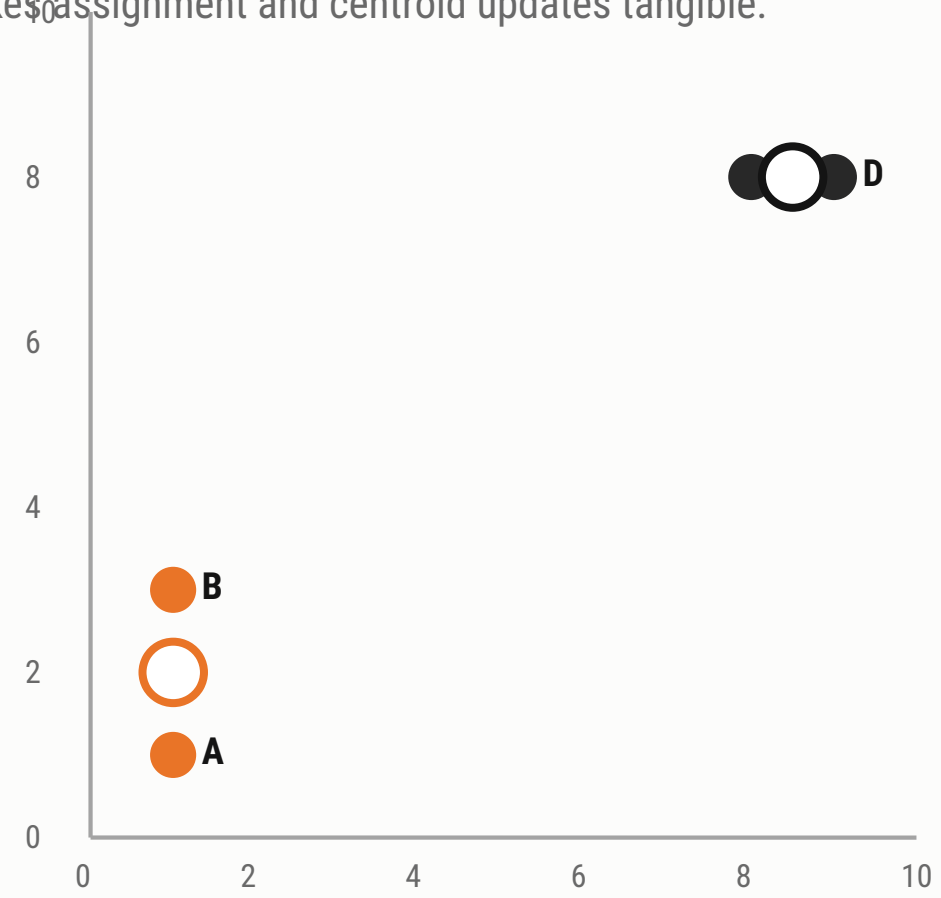
Points A(1,1), B(1,3), C(8,8), D(9,8). Initial centers at (1,1) and (9,8).

Point	Closest center	Assigned cluster
A(1,1)	(1,1)	1
B(1,3)	(1,1)	1
C(8,8)	(9,8)	2
D(9,8)	(9,8)	2

Update means Cluster 1 center → (1,2). Cluster 2 center → (8.5,8). Reassign and repeat.

See the clusters—not just their labels

A 2D plot makes assignment and centroid updates tangible.



Updated centroids

Orange: (1, 2)
Dark: (8.5, 8)

Outlines mark means.

When does K-Means perform poorly?

Clustering is an assumption about geometry, not an oracle that discovers “true categories”.

Choose K

How many clusters should there be?

Feature scale

Tempo 200 vs energy 0–10 may dominate distance.

Shape

Elongated or nested groups may not fit K-Means.

Initialization

Different initial centers can yield different results.

Good habit Scale numerical features when appropriate; visualize the results; avoid interpreting clusters as objective truth.

One artificial neuron as a threshold gate

For binary inputs, set output = 1 if $w_1x_1 + w_2x_2 + b \geq 0$, otherwise 0.

$$z = w_1x_1 + w_2x_2 + b$$

Output = step(z)

x_1	x_2	z, with w=[1,1], b=−0.5	OR
0	0	−0.5	0
0	1	0.5	1
1	0	0.5	1
1	1	1.5	1

The bias shifts the decision threshold; this example implements the Boolean OR gate.

Perceptron weights change after mistakes

Update $w \leftarrow w + \eta(y-\hat{y})x$; update $b \leftarrow b + \eta(y-\hat{y})$.

Given	Value
Input x	(1, 0)
Correct class y	1
Current w, b	(0, 0), −1
Prediction	0
Learning rate η	0.5

One training update

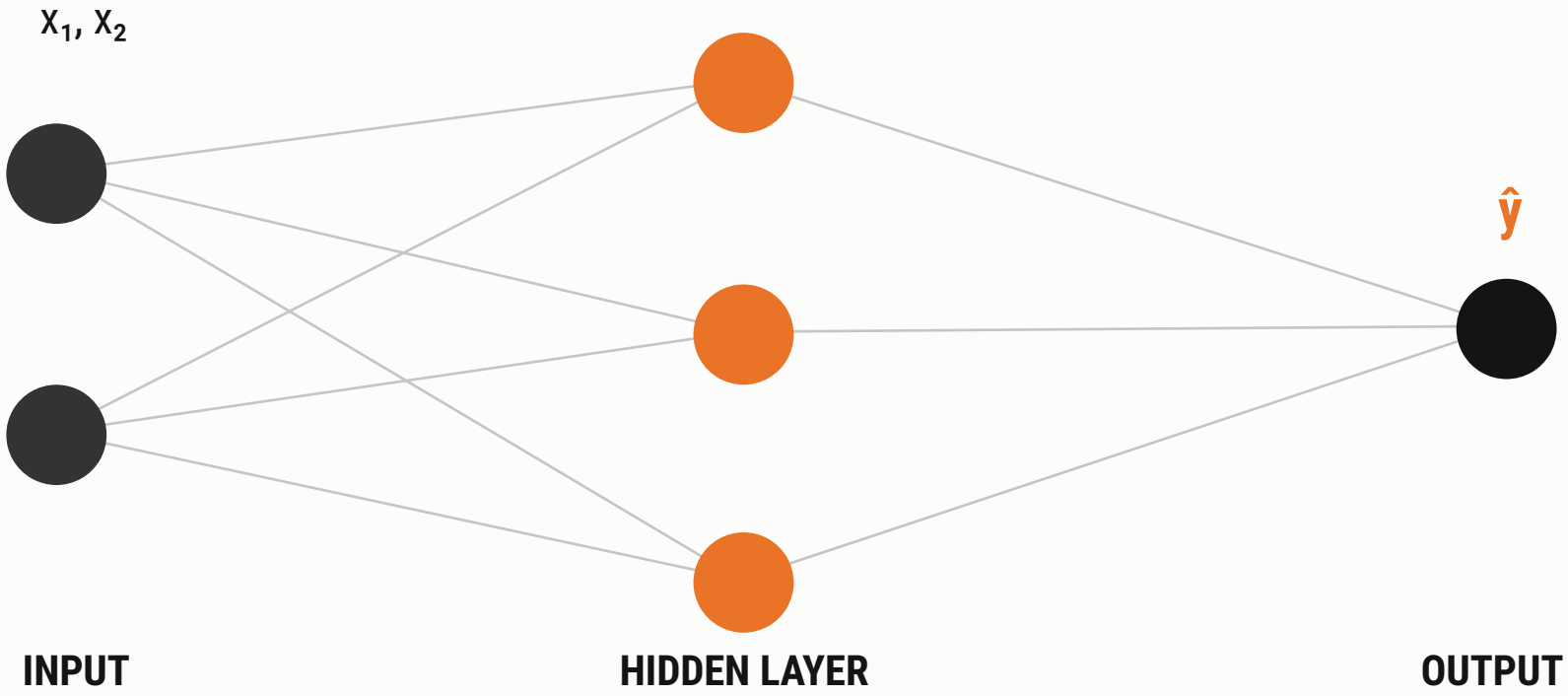
Error = 1 − 0 = 1
$\Delta w = 0.5 \times 1 \times (1,0) = (0.5,0)$
$\Delta b = 0.5$

New w = (0.5,0), b = −0.5.
New z = 0 → predict class 1.

The update fixes this example; many passes over many examples may still be necessary.

From one neuron to layers of computation

Weights, biases, nonlinear activations, and trainable representations.



Calculate one neuron’s output

Input $x=(1,0)$, weights $w=(0.6,-0.4)$, bias $b=-0.1$; use sigmoid.

Weighted sum

$z=0.6\cdot1-0.4\cdot0-0.1$

→

Activation

$\sigma(z)=1/(1+e^{-z})$

→

Output

$\sigma(0.5)\approx0.622$

Interpretation

With threshold 0.5, classify as 1. A numeric sigmoid output is not automatically a calibrated probability.

How do weights get better?

A repeated optimization loop, not a magic act of understanding.

Forward → Compute prediction

Loss → Measure error

Backward → Compute gradients

Step → Update weights

```
for batch in training_data:
    prediction = model(batch.x)
    loss = criterion(prediction, batch.y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

PEDAGOGICAL EVENT 03

Match tasks to algorithms

Different tasks require different assumptions and models.

PREDICT FIRST

A. Predict a price B. Group songs C. Recognize a digit
D. Search a sorted list

Choose among regression, K-Means, neural network, binary search.

GROUP CHALLENGE • 2 min

One reasonable choice for each task

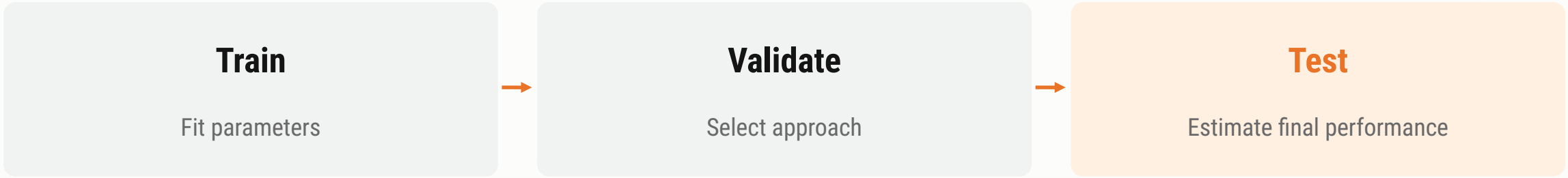
Some tasks admit multiple strong algorithms, not one uniquely correct answer.

Task	One fitting method	Why
Predict price	Regression	Numeric target
Group songs	K-Means	Clusters without labels
Recognize a digit	Neural network	Learn patterns from images
Search sorted list	Binary search	Explicit divide-and-conquer procedure

Important Neural networks can also do regression; decisions depend on data, cost, interpretability, and accuracy.

Train, validate, then test

Performance on memorized training data is not evidence of useful predictions.



Avoid leakage Split appropriately first; fit preprocessing only on the training subset. For time series, respect time order.

Read a confusion matrix

Example: 20 predictions, 10 genuinely positive cases.

	Predicted positive	Predicted negative
Actually positive	TP = 8	FN = 2
Actually negative	FP = 3	TN = 7

Accuracy = 15/20 = 75% **Precision = 8/11 ≈ 73%** **Recall = 8/10 = 80%**

PEDAGOGICAL EVENT 04

Can 95% accuracy be a terrible model?

A rare-condition classifier sees 100 examples: 95 negatives, 5 positives.

PREDICT FIRST

The model always predicts “negative.” Its accuracy is 95%. Is it useful?

Calculate recall for the positive cases and explain why accuracy is deceptive.

Underfitting and overfitting

Complexity is useful only when it helps with unseen examples.

Underfit

Model too constrained to capture important relationships.

Poor training + test results.

Good fit

Captures useful signal that generalizes.

Similar, acceptable train + test error.

Overfit

Memorizes noise and chance details.

Training great, test disappointing.

Countermeasures More representative data, cross-validation, regularization, simpler models, and leakage checks.

A technically working model can still fail people.

Ask: where did the examples come from, and how will errors affect users?

Representativeness

Who is missing from training data?

Privacy

What data is necessary and permitted?

Fairness

Does performance differ across groups?

Security

Can inputs or outputs be manipulated?

Classroom principle Use synthetic or permissioned datasets. Treat predictions as fallible, not unquestionable facts.

How an ML project is built

Separate the data layer, algorithms, training framework, evaluation, and deployment.

What each Python tool is actually for

Libraries solve different engineering jobs; they do not compete as identical alternatives.

Tool	Role	Concrete use
Python	Language / orchestration	Interpret logic, scripts
NumPy	Efficient numerical arrays	Vectors, matrices
pandas	Tabular data	CSV loading, cleaning
Matplotlib	Visualization	Scatter plots, loss curves
scikit-learn	Classical ML	K-Means, trees, regression
PyTorch	Deep learning / autograd	Neural networks, tensors

Alternatives exist: e.g., Polars for dataframes, TensorFlow/JAX for numerical deep learning.

What is a notebook, virtual environment, and Git?

These are engineering tools around ML—not classifiers or training algorithms.

Notebook / Jupyter

Run code in cells; view plots and tables; experiment quickly.

Virtual environment

Keep project dependencies isolated and reproducible.

Git / GitHub

Track code changes, collaborate, and document what ran.

```
python -m venv .venv
python -m pip install numpy pandas matplotlib scikit-learn
```

Activate the environment before installing. Notebook use is optional; VS Code and Google Colab are alternative front ends.

Training and predicting in real Python

Example: one-feature regression (temperature → sales).

```
from sklearn.linear_model import LinearRegression

X = [[10], [20], [30]]    # each row is 1 sample
y = [15, 25, 35]          # target values

model = LinearRegression()
model.fit(X, y)             # estimate coefficients
print(model.predict([[25]]))  # [30.]
```

Explain fit learns coefficients; predict uses them on a new feature vector. This toy example does not assess generalization.

PEDAGOGICAL EVENT 05

Read code like an engineer

What do the following methods mean in an ML training pipeline?

PREDICT FIRST

```
model.fit(X_train, y_train)
model.predict(X_test)
```

Who learns from data? Which data must stay hidden until evaluation?

CODE READING • 90 s

Training is not the same as inference

The fitted model contains coefficients, split rules, or neural parameters.

fit(X_train, y_train)

TRAINING
Uses examples to learn parameters or structure.

E.g., slope w and intercept b.

predict(X_new)

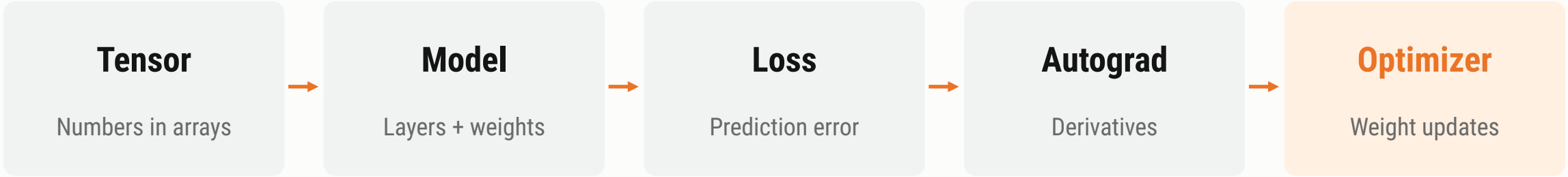
INFERENCE
Applies the trained model to new inputs.

E.g., $\hat{y} = w \times 25 + b$.

Test labels Use y_test to calculate evaluation metrics after producing predictions—not to fit the model.

How PyTorch fits the architecture

Tensors → neural network → loss → autograd → optimizer → trained parameters.



Compare scikit-learn hides many training details behind fit; PyTorch exposes the training loop.

Build a tiny differentiable model

An intentionally short training loop showing the key operations.

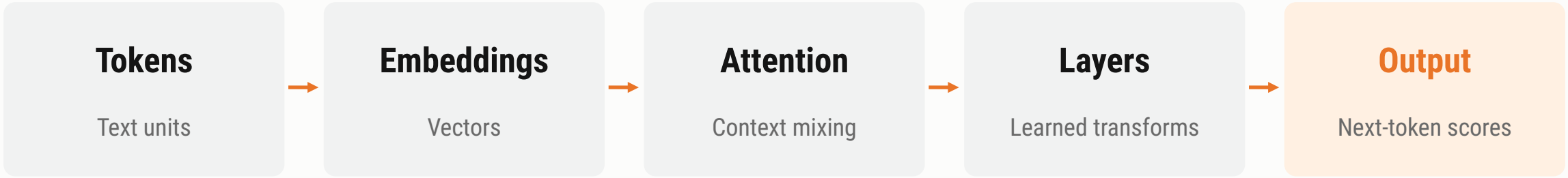
```
import torch
model = torch.nn.Linear(1, 1)
loss_fn = torch.nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.01)

for epoch in range(200):
    pred = model(x_tensor)
    loss = loss_fn(pred, y_tensor)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

x_tensor and y_tensor must already be float tensors of compatible shape [N,1].

Tokens, embeddings and Transformer blocks

A typical language model processes token sequences as learned vector representations.



Technical nuance

Language model training is mostly self-supervised token prediction; chat behavior may involve further fine-tuning.

Transformer architecture: Vaswani et al., Attention Is All You Need (2017).

Context changes what a token should represent

“bank” in “river bank” differs from “bank loan.”

River bank

Nearby words supply evidence for geographic meaning.

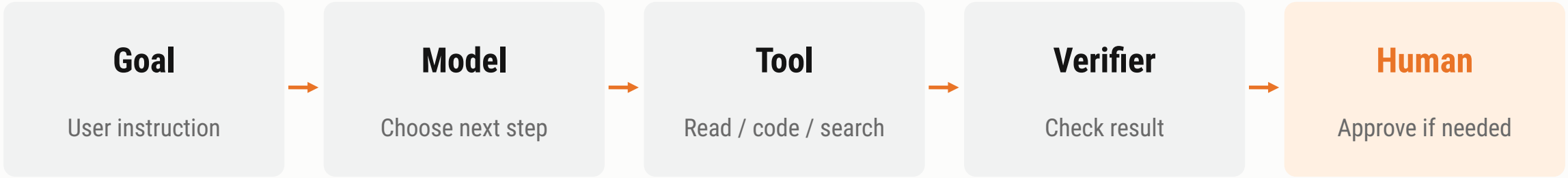
Bank loan

Nearby words supply evidence for financial meaning.

Technical picture Attention forms weighted combinations of token representations using learned query–key similarities and values.

An agent is more than a chat response

A model can plan, call permitted tools, inspect results, and continue under explicit constraints.



Engineering rule Tool access, sandboxing, authentication, logging, and stopping conditions matter as much as prompts.

Four concrete Machine Learning Club projects

Start with clear data and small models. Increase complexity only when the basics are working.

01 • Regression

Ice-cream demand from temperature

scikit-learn

02 • Clustering

Music mood groups: tempo × energy

K-Means

03 • Tree

Explain classification rules from data

DecisionTreeClassifier

04 • Neural network

Recognize handwritten digits

PyTorch / MNIST

Longer term Build a model, explain why it works, test its weaknesses, and present a reproducible demo.

Mini-lab: invent two music clusters

Learners act as the clustering algorithm before seeing any code.

Song	Tempo	Energy
A	65	3
B	72	2
C	142	9
D	135	8

Your assignment

1. Draw a 2D scatter plot.
2. Split songs into K = 2 groups.
3. Compute one average point for each group.
4. Name the groups carefully.

Work in pairs for 5–8 minutes. No previous programming experience needed.

Centroids summarize two discovered groups

Means are computed coordinate-wise.

Group	Members	Mean tempo	Mean energy
Cluster 1	A, B	(65+72)/2 = 68.5	(3+2)/2 = 2.5
Cluster 2	C, D	(142+135)/2 = 138.5	(9+8)/2 = 8.5

Do not overclaim

We found groups in two chosen features, not an objective measurement of what listeners feel.

Sources & learning resources

Official educational references for the factual, mathematical, and code material in this extended deck.

History	The National Museum of Computing — tnmoc.org/bombe
Algorithms / ML	scikit-learn User Guide — scikit-learn.org/stable/user_guide
ML course	Google Machine Learning Crash Course — developers.google.com/machine-learning/crash-course
Neural networks	PyTorch Learn the Basics — docs.pytorch.org/tutorials/beginner/basics/
Transformers	Vaswani et al. (2017), “Attention Is All You Need” — arxiv.org/abs/1706.03762

Charts, tables, and diagrams in this extended deck are newly authored educational illustrations.

If AI can write code,

is learning programming becoming pointless?

Ask students for arguments on both sides before advancing.

no.